EduLLM: A Language Model Trained on Engineering Educational Content

Department of Computer Engineering

Sarvajanik College of Engineering & Technology

Team:-

Vadid Shaikh (**ET23BTCO053**)
Yashesh Patel **(ET23BTCO043)**
Pawankumar Navinchandra (**ET23BTCO046**)

Mentored by Prof. Dr. Keyur Rana

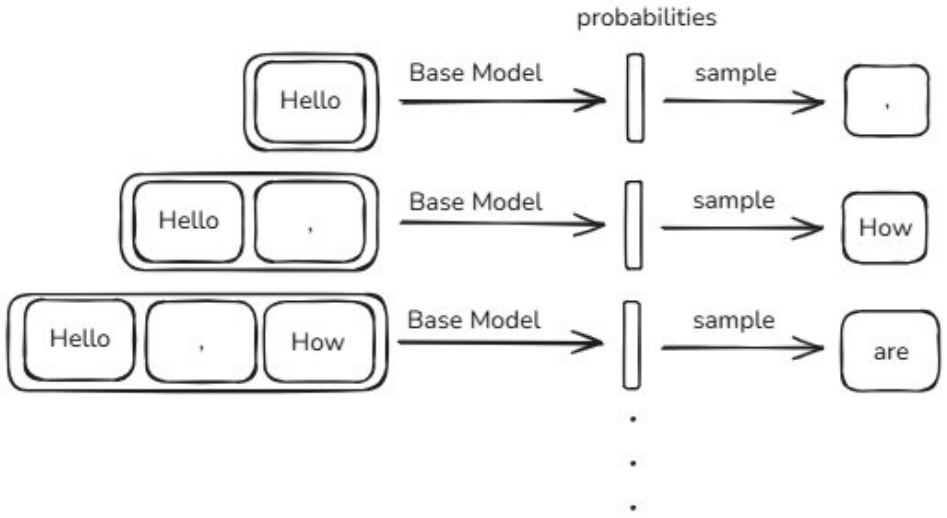
Semester: 5th Semester

Year: 3rd Year

BACKGROUND OF THE PROBLEM

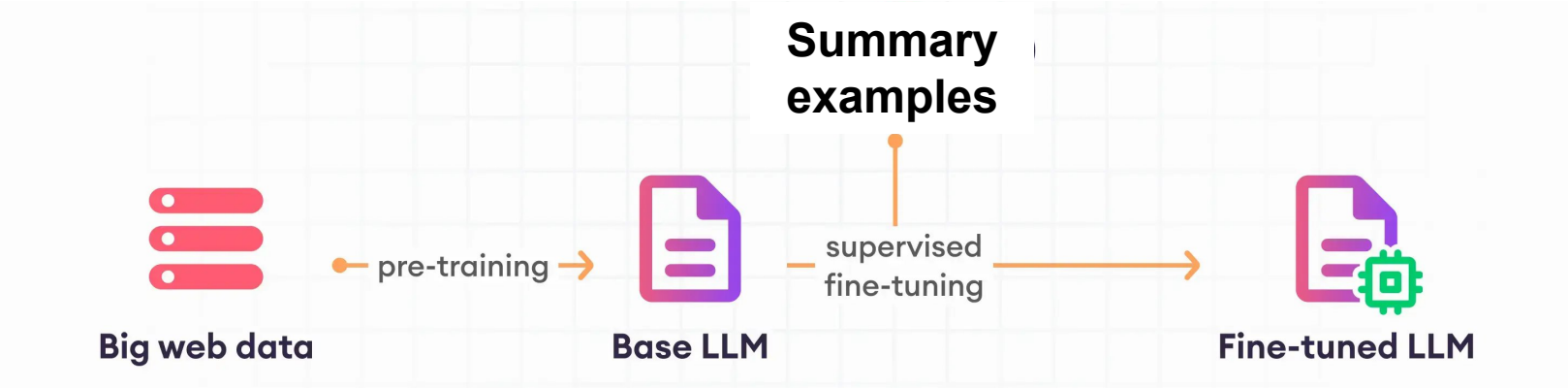
Introduction to LLMs

- LLMs are trained in three stages:-
  - Base Model
  - SFT Model
  - RLHF Model (Out of scope for now)
- The **base model** is trained on a massive text corpus collected from the internet and other sources to **learn next-token prediction** enabling it to acquire **general language patterns, grammar, and world knowledge**, though it is not yet specialized for specific tasks such as conversation, summarization, or reasoning.



Summarization

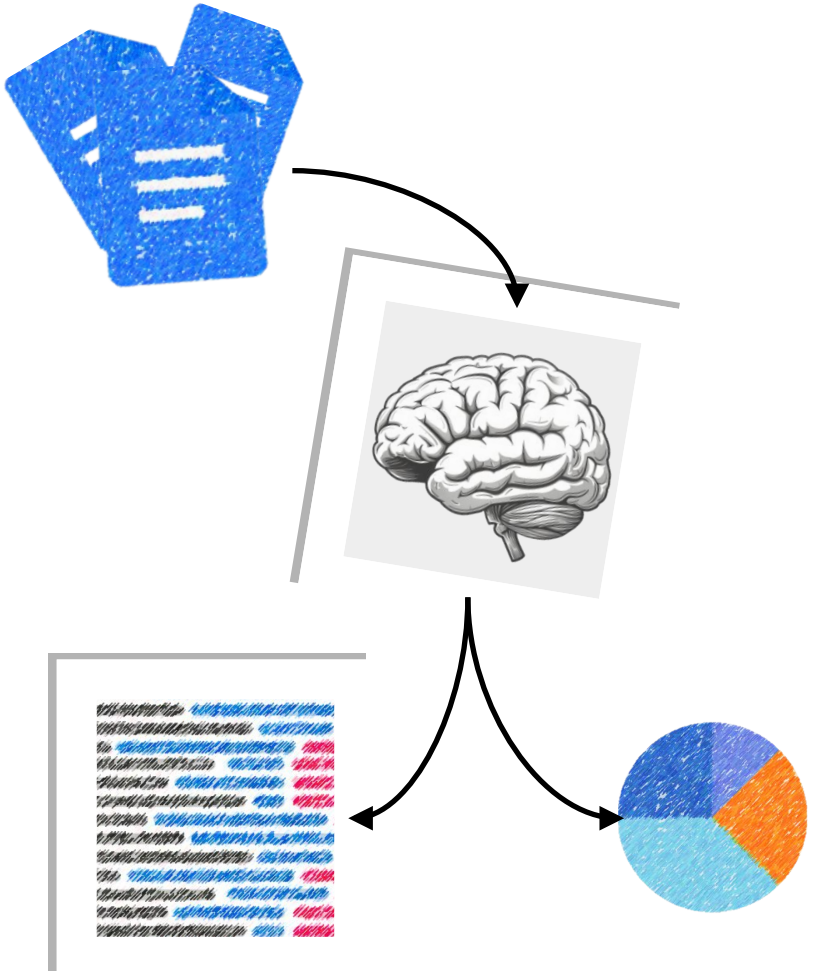
- The **SFT (Supervised Fine-Tuning) model** is trained on a comparatively smaller, high-quality dataset containing examples of conversations between users and assistants.
- This stage **teaches the model how to interact naturally** with users by following instructions and maintaining conversational flow.
- When **trained on datasets where the assistant summarizes passages** provided by users, the **model also learns** to generate effective summaries.



EduLLM

Motivation

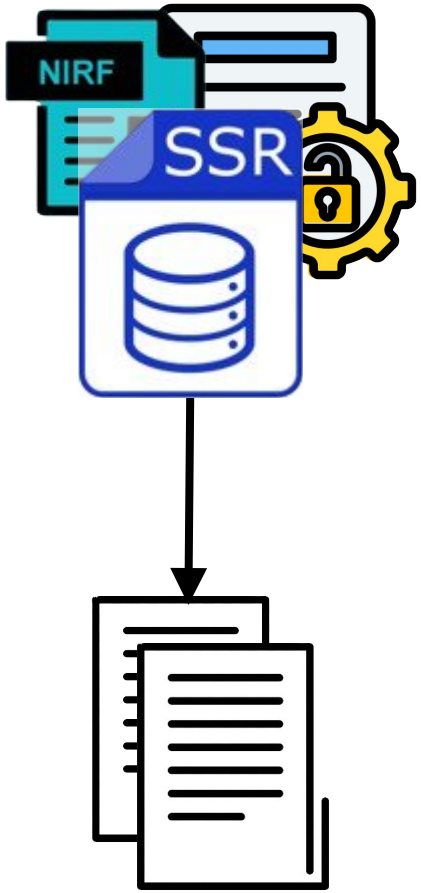
- To build a system for college that retrieves ***relevant document content***, ***summarizes*** it intelligently and ***analyzes numerical data*** providing ***visual charts*** as well.
- Integrating **RAG (Retrieval-Augmented Generation) pipeline** with LLMs helps with these tasks and saves a lot of time.
- Users get **accurate, context-based, and summarized answers** from their own college's document.



EduLLM Use Cases

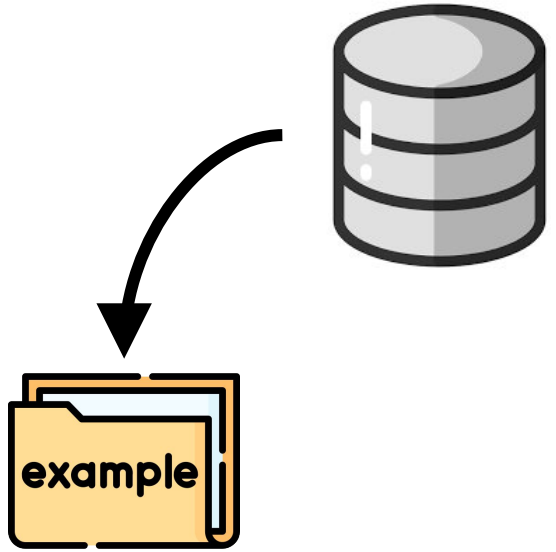
Automatic Summarization of **Accreditation Documents**

- **Mandatory Disclosure, NIRF, SSR etc. documents**
- Summarizes lengthy documents like SSR, AQAR, or MoUs into section-wise or metric-wise briefs.
- No need to browse hundreds of PDFs and Saves hours of manual reading.
- Visual Dashboard for Analysis & Reporting - Answers and data extracted by EduLLM can be displayed as **charts, tables, and infographics.**



Examples

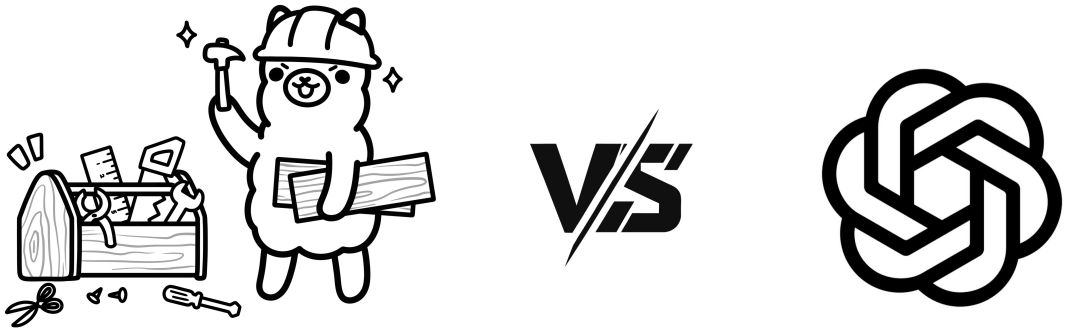
- **Summarization:-**
  - Give a **200-word summary** of Criterion 2 of our latest AQAR.
  - **Summarize the research output** trend for NIRF metrics.
  - **List all MoUs** related to **student internships**.
- **Counting Problems:-**
  - What are the **student-teacher ratios** for the last 5 years?
  - **How many MoUs** were signed with industries in 2023?
- **Charting Tasks:-**
  - **Bar chart of research publications** year-wise.
  - **Pie chart of faculty PhDs** by department.



RESEARCH GAPS

Local vs API-Based

- Now, You might suggest using an online API from ChatGPT or other LLMs, but this approach poses a risk to **data confidentiality**. A **local system** can eliminate this problem by keeping our data within a secure environment, ensuring that sensitive information never leaves the organization's infrastructure.



Model/Data Creation

- **Creating a dataset** for fine-tuning is not always feasible, as we cannot simply **hire many professionals** like OpenAI did to produce high-quality conversational data.
- Even if we manage to generate such examples, the process would be expensive and require manual review to **filter out low-quality or irrelevant data**.
- Using publicly available datasets also poses challenges, as building an **effective model from scratch** using them is extremely difficult.
- Finally, the overall **infrastructure and training costs** would be very high, since achieving a well-performing model typically requires multiple training iterations.

PROBLEM STATEMENT

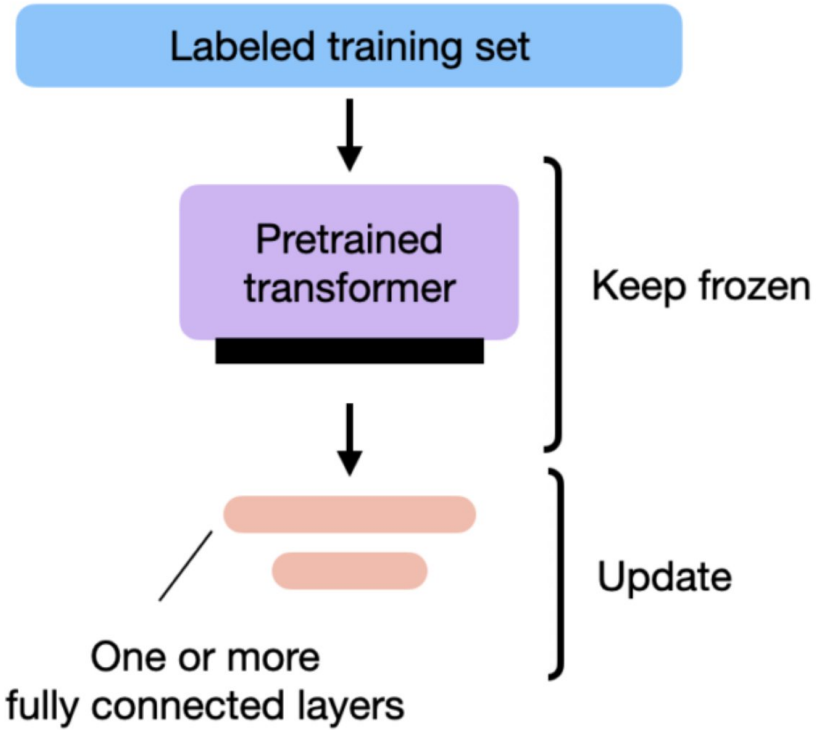
Our Aim!

- **Challenge 1 (Data Confidentiality) :-** Using external, API-based LLMs (like ChatGPT) to process this data is not a viable option. It poses a significant risk to data confidentiality, as sensitive institutional information would have to leave the organization's secure infrastructure.
- **Challenge 2 (Cost and Feasibility) :-** The alternative, building a custom-trained model from scratch, is not feasible for an institution. It involves prohibitively high costs for data creation, manual review, and model training.
- **The Core Problem:-** How can we create an intelligent system to retrieve, summarize, and analyze our college's private documents **securely** and **cost-effectively**, without the risks of third-party APIs or the high expense of full model training

PROPOSED SOLUTION

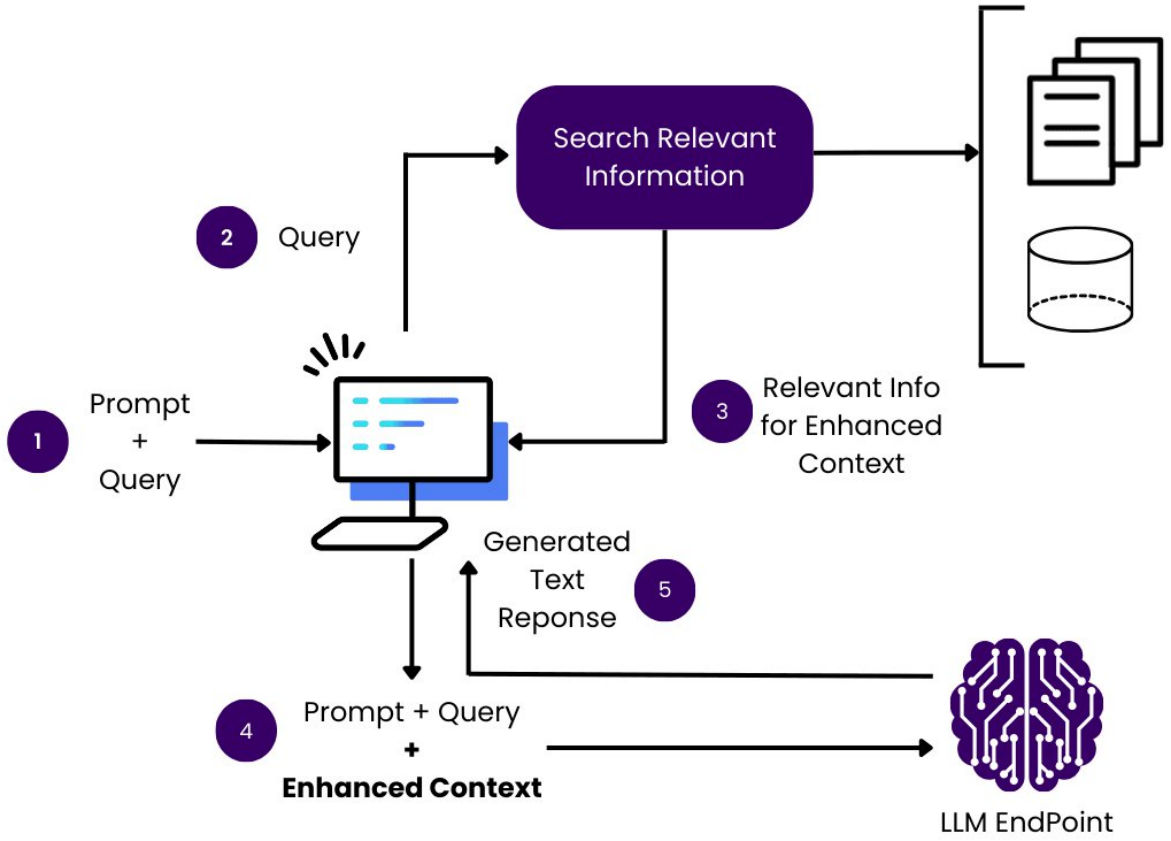
Transfer Learning

- We can either use **transfer learning** by freezing some layers and fine-tuning the rest of the model or apply the **RAG** approach (discussed on the next slide).
- However, using transfer learning again introduces the issue of **high training costs**. Another major drawback is that the fine-tuned model would only store document information in its **internal memory**, which might omit crucial details, potentially leading to **misinterpretation or errors** in important tasks.



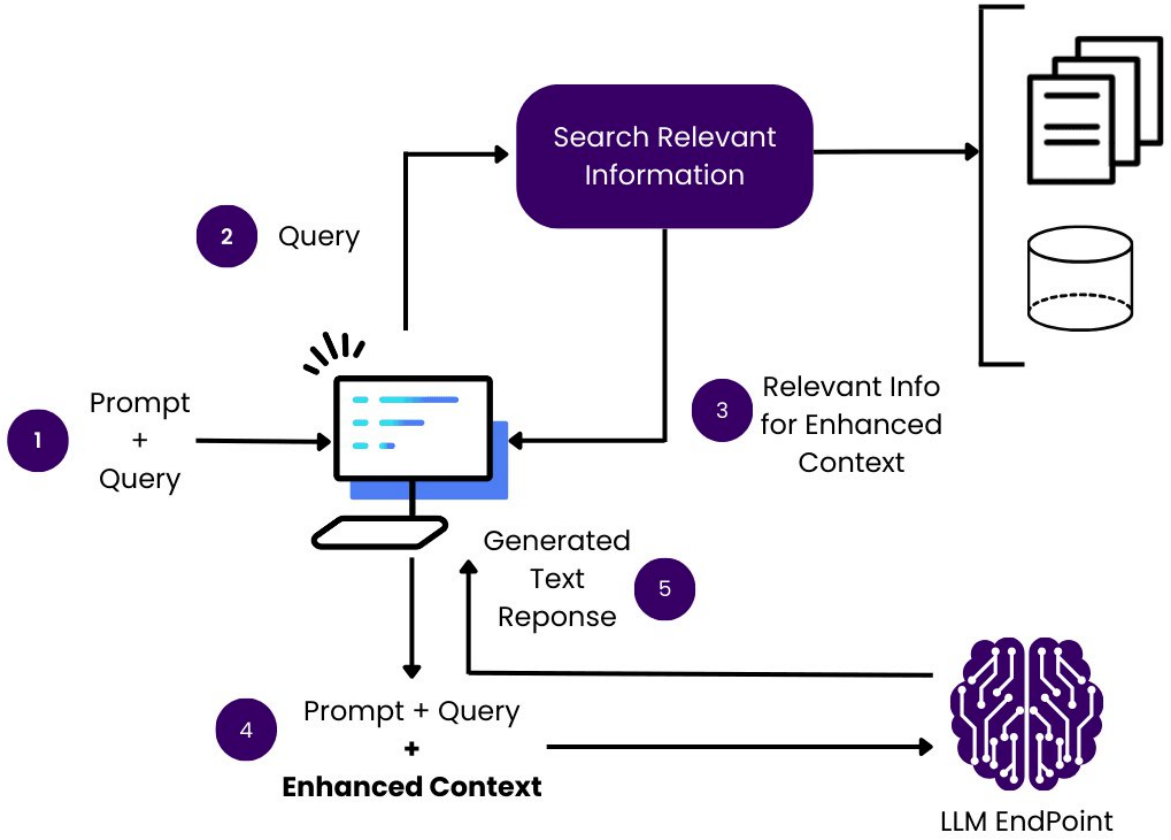
Introduction to RAG

- **Retrieval-Augmented Generation (RAG)** is an advanced technique used to enhance the performance of Large Language Models (LLMs) by combining **information retrieval** with **text generation**.
- Instead of relying solely on the model's internal knowledge (which is fixed after training), RAG allows the model to **dynamically fetch relevant information** from external data sources such as documents, databases, or knowledge bases at query time.



Introduction to RAG

- This approach helps the model generate more **accurate, up-to-date, and context-specific** responses, especially for tasks like
  - **document summarization**
  - **question answering**
  - **knowledge-based reasoning**
- It also **enables retrieval from secure, private document stores** without exposing sensitive information to external APIs.



Hardware Requirements (Est.)

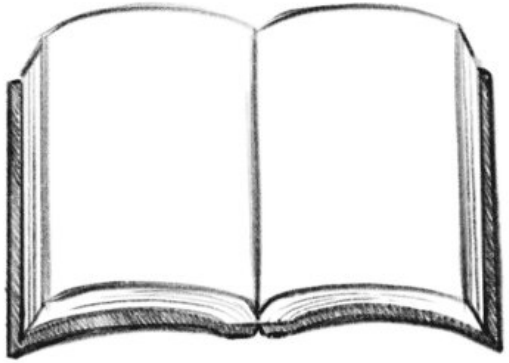
Component	You Proposed	Estimated for 25 concurrent users
GPU	Gigabyte RTX 4090 24 GB (₹2 lakhs - ₹2.5 lakhs)	Plenty for quantized 7B/8B models. We can even serve **2 models** concurrently (e.g., one for reasoning, one for code).
CPU	16 core (₹50,000 - ₹70,000)	LLM inference is GPU-bound but retrieval and preprocessing (tokenization, RAG pipeline) benefit from extra cores.
RAM	2 × 32 GB = 64 GB total (₹20,000 - ₹30,000)	Ideal. Gives headroom for vector DB, caching, and model orchestration (LangChain / FastAPI).
Storage	1 TB NVMe SSD (₹10,000 - ₹15,000)	Perfect — fast I/O for embeddings, docs, logs.

Future Scope

- To explore additional available options and implement the model using a **RAG pipeline**. One of our primary concerns will be to, **optimize for speed and accuracy** to minimize errors during inference.
- **Improving retrieval quality** by experimenting with different vector stores, embeddings, or ranking algorithms.
- **Scalability and deployment optimization** for handling large document collections efficiently.
- To design a **tier-based access control system** for **professors and students** to manage and restrict data access according to their respective roles and privileges.

Bibliography

- Deep Learning Series by Andrej Karpathy
- Build a Large Language Model (From Scratch) by Sebastian Raschka
- Training language models to follow instructions with human feedback by OpenAI
- Chain-of-Verification Reduces Hallucination in Large Language Models by Meta



Thank YOU